Ingeniería y selección de variables

Unidad 3 · Creación de variables, filtros, envoltura (RFE) e integrados (Lasso)

Razones y polinomios · Varianza · Información mutua · RFE · Lasso

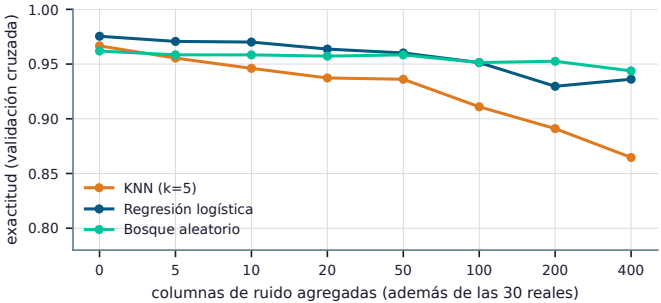
Cáncer de mama de Wisconsin · scikit-learn

Plan de la unidad (≈45 min)

Qué veremos

- **A. Crear** variables: razones del dominio, transformaciones, polinomios e interacciones
- **B. Seleccionar** variables: tres familias de métodos
- **C.** Métodos de **filtro**: umbral de varianza, correlación e información mutua
- **D.** Métodos de **envoltura**: eliminación recursiva de variables (RFE)
- **E.** Métodos **integrados**: selección con Lasso
- **F.** Comparación de métodos, estabilidad y receta

¿Por qué trabajar las variables? Más columnas no siempre es mejor



Diagnóstico de cáncer de mama (30 variables reales). Se agregan columnas de **ruido puro** y se mide la exactitud con validación cruzada (5 particiones, 3 repeticiones).

Lo que dice la gráfica

- **KNN** cae de 0.967 a 0.865 con 400 columnas inútiles: las distancias se llenan de ruido.
- La **regresión logística** baja de 0.975 a 0.936; el **bosque**, de 0.962 a 0.944.

Idea de la unidad

Hay que **construir** buenas variables y **descartar** las que sobran. Cada variable inútil cuesta exactitud, tiempo y claridad.

Nuestro conjunto de datos médico: cáncer de mama de Wisconsin

Medida del núcleo celular	Qué describe
radio, perímetro, área	tamaño del núcleo
textura	variación de los tonos de gris
suavidad, compacidad	regularidad del contorno
concavidad, puntos cóncavos	hendiduras del contorno
simetría, dimensión fractal	forma global y rugosidad

Cada una de las 10 medidas se resume de 3 formas: **media** (`mean`), **error estándar** (`error`) y **peor valor** (`worst`). En total, 30 variables.

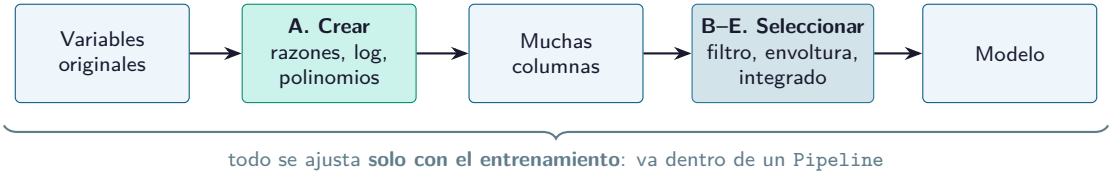
Los datos

- $n = 569$ pacientes (imágenes de biopsia por aspiración con aguja fina).
- Objetivo: **maligno** (212) o **benigno** (357).
- Ya viene incluido en scikit-learn (`load_breast_cancer`).

Punto de partida

Regresión logística con las 30 variables estandarizadas: exactitud de **0.975** en validación cruzada. ¿Podemos lograr algo parecido con muchas menos mediciones?

Mapa de la unidad: crear y seleccionar, siempre dentro del pipeline



Crear (A)

Agrega información que el modelo no puede construir por sí solo: una razón, una potencia, un producto entre variables.

Seleccionar (B–E)

Se queda con las variables que *aportan*: menos sobreajuste, menos costo y un modelo que se puede explicar.

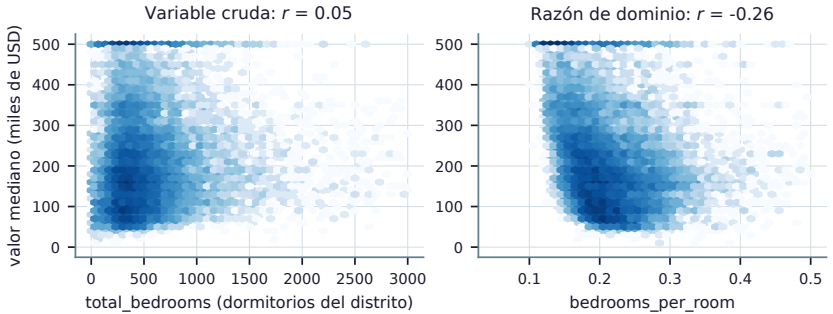
A1 · Ingeniería de variables: codificar el conocimiento del dominio

Tipo	Ejemplo	Por qué ayuda
Razón o tasa	habitaciones por hogar; índice de masa corporal (IMC = peso/talla^2)	compara en una escala común; quita el efecto del tamaño
Diferencia o cambio	presión después menos presión antes	lo importante suele ser el cambio, no el nivel
Agregado	promedio de compras por cliente en 30 días	resume muchas filas en una sola variable
Fecha o tiempo	día de la semana, mes, hora	revela ciclos que la fecha cruda esconde
Transformación	$\log x$, $\sqrt{x}$	reduce asimetría y atípicos
Discretización	edad en rangos (niño, adulto, mayor)	capta efectos por tramos
Interacción	$x_1 \cdot x_2$	el efecto de una variable depende de otra

La pregunta clave

¿Qué sabe un experto que los números crudos no dicen? Cada variable nueva es una **hipótesis**: se acepta si mejora la validación cruzada.

A2 · Ejemplo: razones en California Housing



Cada punto es un distrito (la escala de color indica cuántos). Eje y: valor mediano de la vivienda. **Izquierda**: dormitorios totales del distrito. **Derecha**: dormitorios por habitación.

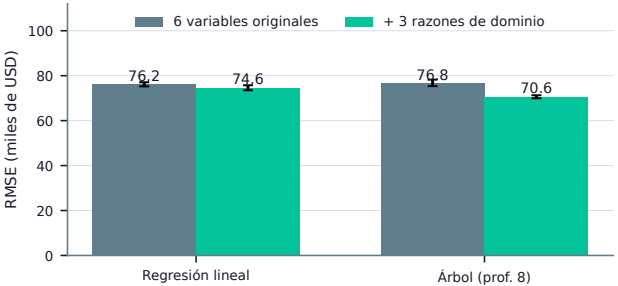
El conteo mide el tamaño del distrito

`total_bedrooms` casi no se relaciona con el precio ($r = 0.05$): un distrito grande tiene muchos dormitorios, caro o barato.

La razón mide cómo son las viviendas

`bedrooms_per_room` = `total_bedrooms`/`total_rooms` sí se relaciona ($r = -0.26$): menos proporción de dormitorios, más casas amplias y caras. Otras razones: `rooms_per_household`, `population_per_household`.

A3 · ¿Mejoran el modelo las razones?



RMSE de validación cruzada (5 particiones), en miles de USD; menor es mejor. Originales: ingreso, edad, habitaciones, dormitorios, población y hogares. Se agregan 3 razones.

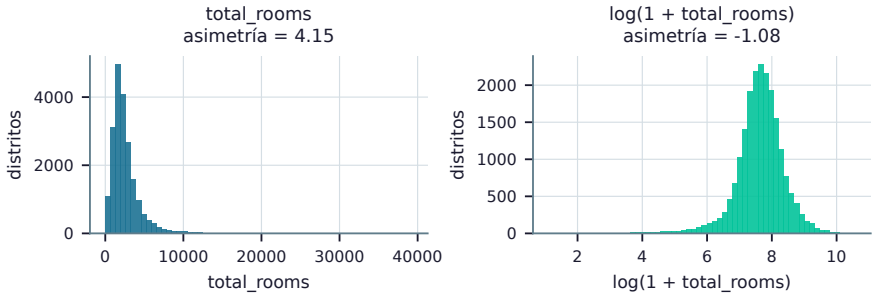
Lo que se ve

- **Árbol**: de 76.8 a **70.6** (8 % menos de error): parte el espacio usando las razones.
- **Regresión lineal**: de 76.2 a 74.6, poca mejora; las razones le dan lo que no puede calcular (dividir una columna entre otra).

Regla

Una variable nueva no garantiza mejora. **Se prueba con validación cruzada**, como cualquier otra decisión.

A4 · Transformaciones: acercar variables asimétricas a la simetría



Histogramas de `total_rooms` (distritos de California) sin transformar (izquierda) y con $\log(1+x)$ (derecha).

Qué cambia

La asimetría baja de 4.15 a -1.08 (en valor absoluto, de 4.15 a 1.08): la cola derecha larga se comprime y los atípicos dejan de dominar.

Cuándo usarla

Variables positivas con cola larga (conteos, ingresos, precios), para modelos lineales y de distancia. Con ceros o negativos: `PowerTransformer` (Yeo-Johnson).

A5 · Variables polinomiales: potencias y productos

Idea

Para que un modelo *lineal* ajuste curvas, se le dan como columnas las potencias y los productos de las variables. Con dos variables x_1, x_2 y grado 2:

$$x_1,\ x_2,\ x_1^2,\ x_1x_2,\ x_2^2$$

El modelo sigue siendo lineal **en los coeficientes**:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_1 x_2 + \beta_5 x_2^2$$

En scikit-learn

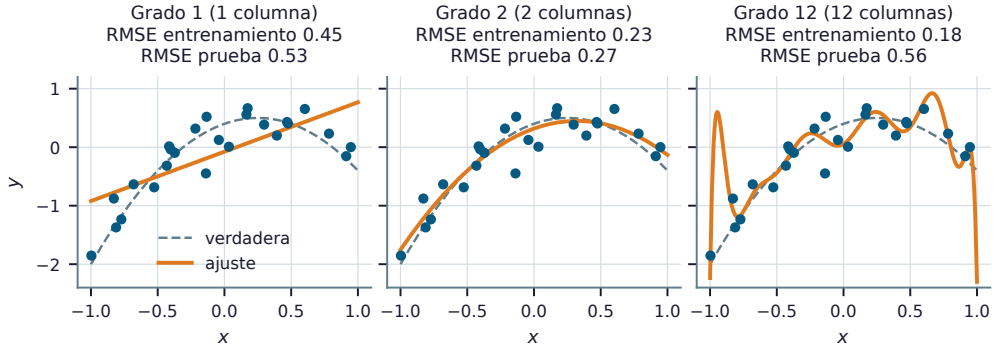
```
PolynomialFeatures(degree=2,
  include_bias=False)
```

- `interaction_only=True`: solo productos entre variables distintas, sin x_j^2.
- Conviene **estandarizar** antes (potencias de números grandes explotan).

Más columnas, más riesgo

El grado controla la flexibilidad; con grado alto el modelo memoriza el ruido.

A6 · El grado del polinomio: flexibilidad contra sobreajuste



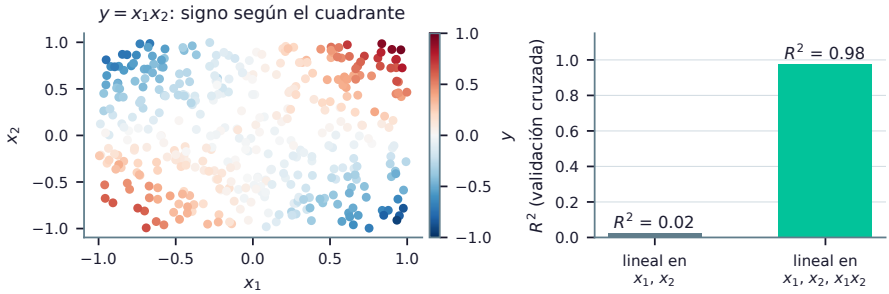
25 puntos de entrenamiento (azul) generados con una parábola más ruido (desviación 0.25); línea discontinua = relación verdadera. RMSE de prueba medido con 400 puntos nuevos.

Grado 1: recta. No capta la curva (sesgo): RMSE de prueba 0.53.

Grado 2: coincide con la forma real. RMSE de prueba 0.27, cerca del piso del ruido (0.25).

Grado 12: pasa por los puntos (entrenamiento 0.18), pero oscila: prueba 0.56.

A7 · Interacciones: cuando el efecto de una variable depende de otra



Datos simulados con $y = x_1x_2$ más ruido. Izquierda: color = valor de y según (x_1, x_2). Derecha: R^2 de una regresión lineal, sin y con la columna x_1x_2.

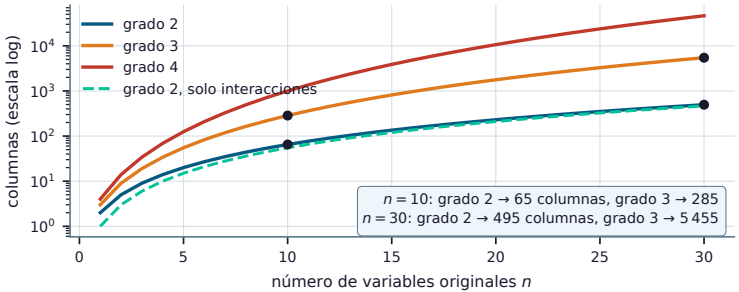
Qué pasa

y es positiva cuando x_1 y x_2 tienen el mismo signo y negativa si no. Ninguna de las dos variables, por separado, predice y: $R^2 = 0.02$. Con el producto, $R^2 = 0.98$.

Ejemplos reales

Dosis × peso del paciente; precio × estación del año; ingreso × zona. Los árboles encuentran interacciones solos; los modelos lineales necesitan que se las demos.

A8 · La explosión de columnas



Cuántas columnas se generan

Con n variables y grado d (sin la columna constante): $\binom{n+d}{d} - 1$.

Con 10 variables: 65 columnas de grado 2 y 285 de grado 3. Con 30 variables: 495 y 5 455.

Cómo controlarlo

Grado bajo (2 o 3), `interaction_only=True`, estandarizar, y **seleccionar** después (secciones siguientes).

A9 · Crear variables dentro del pipeline

```python
import numpy as np
from sklearn.preprocessing import FunctionTransformer, PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression

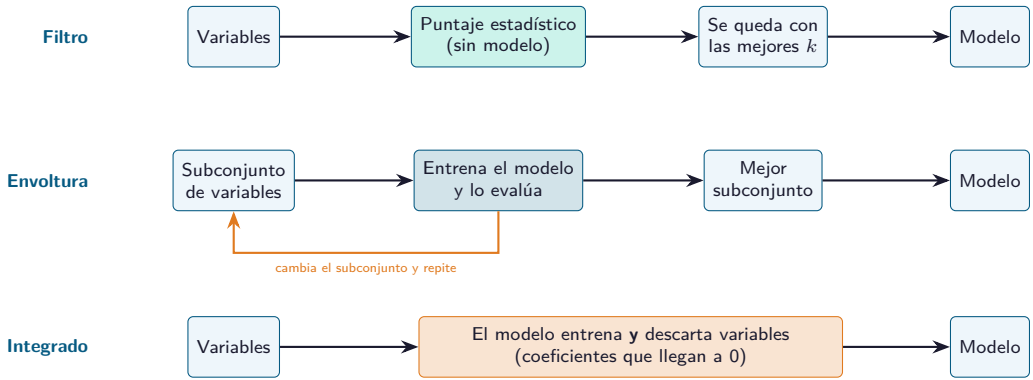
def agrega_razones(df):                          # variables de dominio
    df = df.copy()
    df["rooms_per_household"] = df["total_rooms"] / df["households"]
    df["bedrooms_per_room"]   = df["total_bedrooms"] / df["total_rooms"]
    return df

crea = FunctionTransformer(agrega_razones)     # lo envuelve como un paso del Pipeline

modelo = make_pipeline(
    crea,
    StandardScaler(),
    PolynomialFeatures(degree=2, include_bias=False),   # polinomios y productos
    LinearRegression(),
)
# modelo.fit(X_ent, y_ent)   # todo se ajusta solo con el entrenamiento
```

Las razones no *aprenden* nada de los datos, así que no hay fuga; pero meterlas en el `Pipeline` garantiza que se calculen igual en entrenamiento, validación y producción.

B1 · Tres familias de selección de variables



La diferencia está en *quién decide* qué variables sobran

Un puntaje estadístico (filtro), una búsqueda que prueba subconjuntos con el modelo (envoltura) o el propio entrenamiento del modelo (integrado).

B2 · Comparación de las tres familias

	Filtro	Envoltura	Integrado
¿Usa un modelo?	No (estadística simple)	Sí, muchas veces	Sí, uno solo
Costo	Muy bajo	Alto	Medio
¿Ve interacciones?	No (cada variable sola)	Sí	Según el modelo
Riesgo principal	Deja variables redundantes y ve una a la vez	Cuesta y puede sobreajustar la selección	Depende de un hiperparámetro y de la escala
En scikit-learn	`VarianceThreshold`, `SelectKBest`	`RFE`, `RFECV`	`SelectFromModel`, Lasso

En esta unidad

Veremos las tres con el mismo problema (30 variables de cáncer de mama) y las compararemos al final con la misma validación cruzada.

C1 · Umbral de varianza: quitar variables que casi no cambian

Idea

Una variable constante no distingue a unos pacientes de otros: no aporta. La varianza mide cuánto cambia:

$$\mathrm{Var}(x) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Se eliminan las variables con $\mathrm{Var}(x) <$ umbral.

Cuándo funciona bien

Columnas constantes o casi (un identificador, una bandera que casi siempre vale 0). Es un primer paso muy barato, que no mira el objetivo.

```python
from sklearn.feature_selection import
    VarianceThreshold

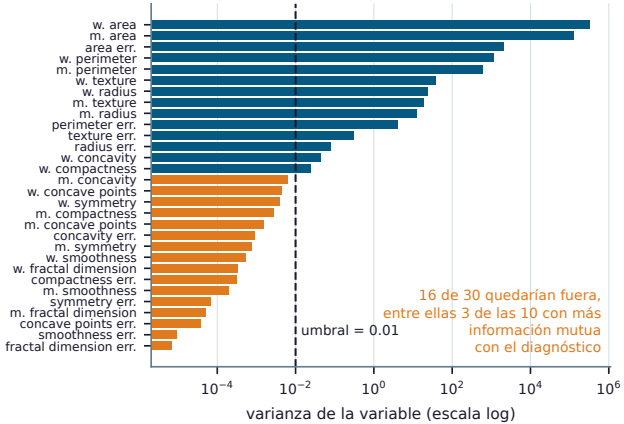
sel = VarianceThreshold(threshold=0.01)
X_sel = sel.fit_transform(X_ent)

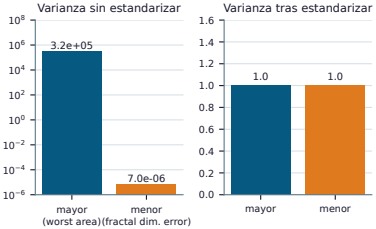
# nombres de las que se quedan:
X_ent.columns[sel.get_support()]
```

Ojo

Para variables 0/1, $\mathrm{Var} = p(1-p)$: umbral 0.01 elimina las que valen 1 en menos del 1 % de los casos.

C2 · La trampa de la varianza: depende de las unidades

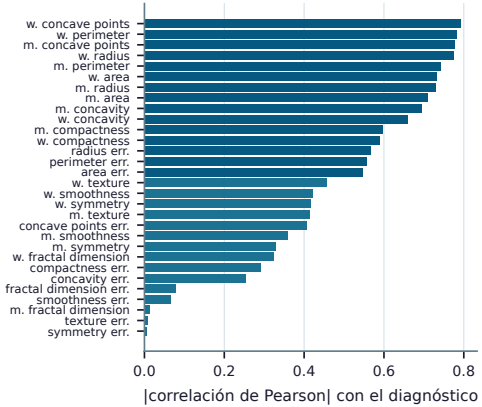




Qué se ve

- Las varianzas van de 7×10^{-6} a 3.2×10^{5}: la unidad de cada variable decide.
- Con umbral 0.01 se descartan **16 de 30**, entre ellas 3 de las 10 con más información sobre el diagnóstico.
- Al estandarizar todas valen 1: el filtro ya no distingue nada.

C3 · Filtro univariado: correlación con el objetivo



Idea

Se calcula, para cada variable x_j, su correlación de Pearson con el objetivo y y se ordena por $|r|$:

$$r_j = \frac{\mathrm{Cov}(x_j, y)}{\sigma_{x_j}\,\sigma_y}$$

Se conservan las k con mayor $|r_j|$ (`SelectKBest` con `f_classif` o `f_regression`, que equivalen a ordenar por $|r|$).

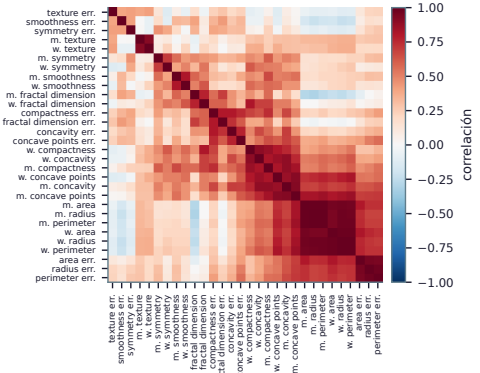
En el diagnóstico

15 variables con $|r| > 0.5$ y 8 con $|r| > 0.7$. Las mejores: `worst concave points` (0.79), `worst perimeter` (0.78), `mean concave points` (0.78). Las peores casi no se relacionan: `symmetry error` (0.007).

Límite

Solo mide relación **lineal** y **una variable a la vez**.

C4 · Redundancia: variables que dicen lo mismo



Qué se ve

Matriz de correlación entre las 30 variables, ordenadas por grupos parecidos. El bloque rojo oscuro: radio, perímetro y área (correlación radio–perímetro 0.998; radio–área 0.987).

Estrategia

De cada par con $|r| > 0.9$ (hay 21 pares de 435), conservar solo **una**: la que más se relacione con el objetivo. Quitar redundantes no pierde información y estabiliza los modelos lineales.

Un filtro por objetivo no basta

Elegir las 5 con mayor $|r|$ con y da varias versiones de lo mismo (radio, perímetro y área del núcleo).

C5 · Información mutua: cuánto «aprendo» de y al conocer x

Definición (variables discretas)

$$I(X;Y) = \sum_{x} \sum_{y} p(x,y) \, \log \frac{p(x,y)}{p(x)\,p(y)}$$

Compara la probabilidad conjunta con la que habría si X y Y fueran independientes. Equivale a la **reducción de incertidumbre**:

$$I(X;Y) = H(Y) - H(Y \mid X)$$

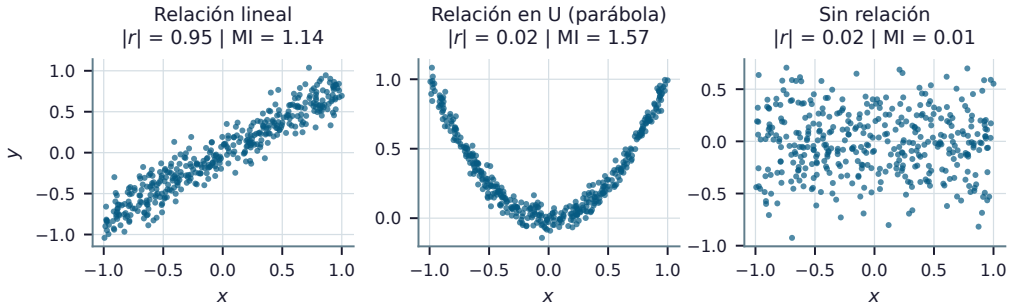
Cómo leerla

- $I(X;Y) \geq 0$.
- $I = 0$ si y solo si X y Y son **independientes**.
- Capta relaciones **no lineales**.
- No tiene signo ni escala fija: sirve para *ordenar*.

En scikit-learn

`mutual_info_classif` y `mutual_info_regression` la estiman con vecinos más cercanos para variables continuas (los valores son estimaciones y cambian un poco entre ejecuciones: fijar `random_state`).

C6 · La correlación no ve todo; la información mutua sí



Tres relaciones simuladas con 400 puntos. Sobre cada panel: |correlación de Pearson| y estimación de la información mutua (en nats).

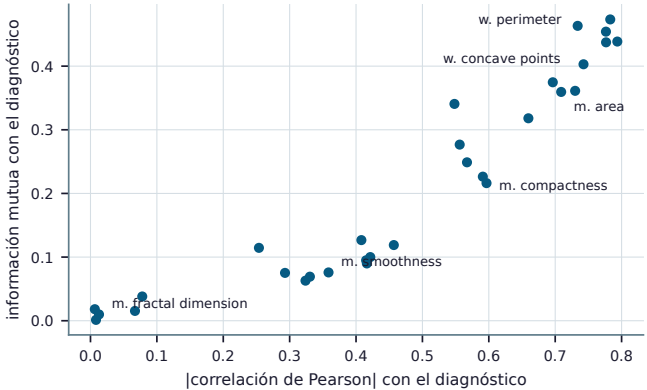
Relación en U

$y = x^2$ depende totalmente de x, pero $|r| = 0.02$: la parte izquierda cancela la derecha. La información mutua (1.57) sí la detecta.

Sin relación

Ambas medidas se acercan a 0 ($|r| = 0.02$ y $I = 0.01$): son las variables que un filtro debe descartar.

C7 · En el diagnóstico: información mutua contra correlación



Qué se ve

Cada punto es una de las 30 variables. Ambas medidas ordenan casi igual (correlación de rangos de Spearman 0.96) y 9 de las 10 mejores coinciden.

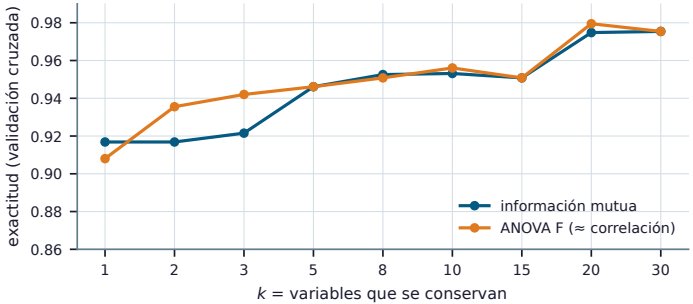
Por qué

Aquí las relaciones son casi monótonas (a mayor tamaño del núcleo, más probable que sea maligno): la correlación basta. La información mutua da ventaja cuando hay relaciones en U o por tramos.

Recomendación

Calcular ambas y comparar es barato.

C8 · Elegir k: validación cruzada con el filtro dentro del pipeline



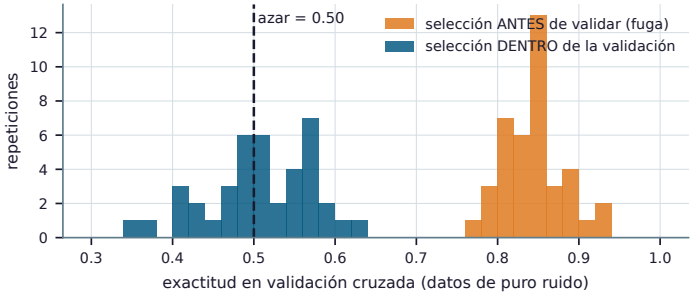
Exactitud con regresión logística según el número k de variables que deja el filtro. 5 particiones $\times$ 3 repeticiones.

Lo que se ve

Con $k = 5$: 0.946; con $k = 10$: 0.953–0.956; con $k = 20$: 0.975–0.979, igual que con las 30 (0.975). Los dos filtros son parecidos.

```
Pipeline([
 ("sc", StandardScaler()),
 ("sel", SelectKBest(
     mutual_info_classif, k=10)),
 ("lr", LogisticRegression())])
```

C9 · Dónde va la selección: dentro de la validación cruzada



Experimento de la Unidad 2: 100 filas, 2 000 atributos de ruido puro y etiquetas al azar.

Seleccionar es *aprender de los datos*

Elegir las mejores variables usa las etiquetas y. Si se hace con **todos** los datos antes de validar, la validación ya vio la respuesta: exactitud de 0.84 con puro ruido (lo correcto es 0.51).

Regla

La selección es un **paso del** `Pipeline`: en cada partición de la validación cruzada se vuelve a elegir con el pedazo de entrenamiento.

D1 · Envoltura: probar subconjuntos con el modelo

Idea

Cada subconjunto de variables se juzga por el desempeño de **un modelo entrenado con él**. Tiene en cuenta las variables *juntas*, no una por una.

Revisar todos es imposible

Con p variables hay 2^p subconjuntos:

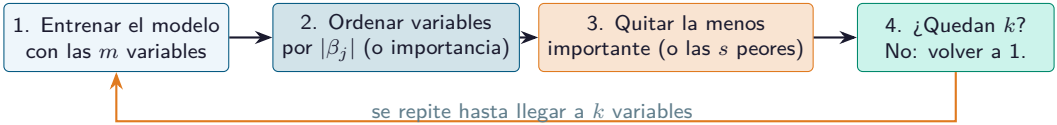
p	subconjuntos
10	1 024
20	1 048 576
30	$\approx 1.07 \times 10^9$

Búsquedas voraces (greedy)

- **Hacia adelante**: empieza sin variables y agrega la que más mejora.
- **Hacia atrás**: empieza con todas y quita la que menos aporta.
- **RFE**: hacia atrás, usando los coeficientes o la importancia del modelo para decidir qué quitar.

No garantizan el mejor subconjunto, pero cuestan unos cuantos entrenamientos.

D2 · RFE: eliminación recursiva de variables



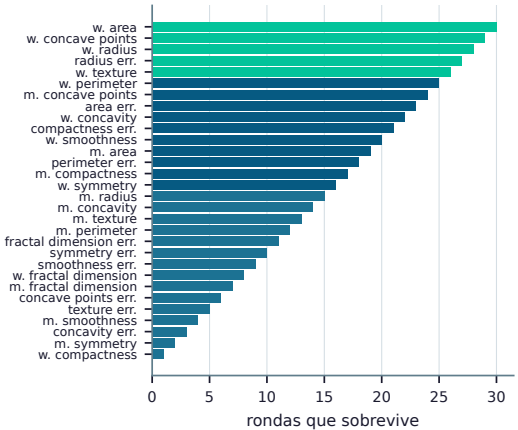
Por qué es recursivo

Al quitar una variable, las demás cambian de importancia (sobre todo si estaban correlacionadas). Por eso **se reentrena en cada ronda** en lugar de ordenar una sola vez.

Requisitos

El modelo debe dar una importancia por variable: `coef_` (modelos lineales, SVM lineal) o `feature_importances_` (árboles). Variables **estandarizadas**, para que los coeficientes sean comparables.

D3 · RFE en el diagnóstico: el orden de eliminación



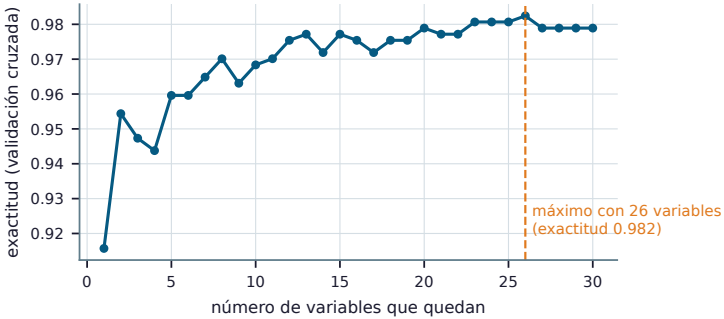
Cómo leerla

RFE con regresión logística (variables estandarizadas), quitando una variable por ronda hasta dejar una. Cada barra indica cuántas rondas sobrevivió: la de arriba se queda hasta el final.

Qué se ve

- Sobreviven: `worst area`, `worst concave points`, `worst radius`, `radius error`, `worst texture`.
- Salen primero: `worst compactness`, `mean symmetry`, `concavity error`, `mean smoothness`.
- Entre las que sobreviven hay medidas de *textura* (distinta del tamaño): RFE no repite lo mismo, a diferencia de un filtro por correlación.

D4 · RFECV: cuántas variables conservar



Exactitud (validación cruzada de 5 particiones) en cada paso de la eliminación, según las variables que quedan.

Qué se ve

Con 1 variable: 0.916; con 5: 0.960; con 10: 0.968. El máximo (0.982) llega con 26 variables; con las 30, 0.979. La curva es plana entre 10 y 30: **unas 10 variables dan casi lo mismo**.

Elegir k

El máximo no siempre es lo mejor: si 10 variables rinden casi igual, el modelo con 10 es más barato y fácil de explicar.

D5 · RFE en código, y sus ventajas y costos

```python
from sklearn.feature_selection import RFE, RFECV
from sklearn.linear_model import LogisticRegression

# k fijo
rfe = RFE(LogisticRegression(max_iter=3000),
          n_features_to_select=10, step=1)

# k elegido por validacion cruzada
rfecv = RFECV(LogisticRegression(max_iter=3000),
              step=1, cv=5, scoring="accuracy")
rfecv.fit(X_est, y)    # X_est: estandarizada
rfecv.n_features_      # numero optimo
X.columns[rfecv.support_]
```

Ventajas

Considera las variables *juntas*; suele dar mejores subconjuntos pequeños que un filtro.

Costos

Entrena unos p modelos (o p/s con `step`$= s$), por cada partición. Depende del modelo elegido y no es estable si hay variables muy correlacionadas.

En un `Pipeline`, `RFE` es un paso más: `("sel", RFE(...))`, antes del estimador final.

E1 · Lasso: penalizar el tamaño de los coeficientes

Regresión Lasso

Ajusta mínimos cuadrados más una penalización sobre la **suma de los valores absolutos** de los coeficientes (norma L1):

$$\min_{\beta} \frac{1}{2n} \sum_{i=1}^{n} \left(y_i - \beta_0 - \textstyle\sum_j \beta_j x_{ij}\right)^2 + \alpha \textstyle\sum_{j=1}^{p} |\beta_j|$$

$\alpha \geq 0$ controla la fuerza: $\alpha = 0$ es mínimos cuadrados; α grande lleva todos los coeficientes a 0.

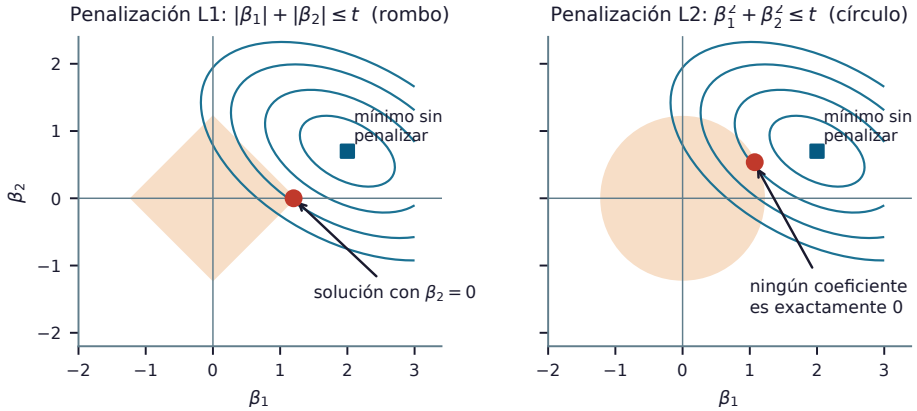
El efecto «selección»

Con la penalización L1, muchos coeficientes valen **exactamente 0**: esas variables salen del modelo. La selección ocurre *mientras se entrena*.

Clasificación

Regresión logística con `penalty="l1"`; el parámetro es `C`$= 1/\alpha$: **menor C, más penalización**.

E2 · ¿Por qué L1 produce ceros? La geometría

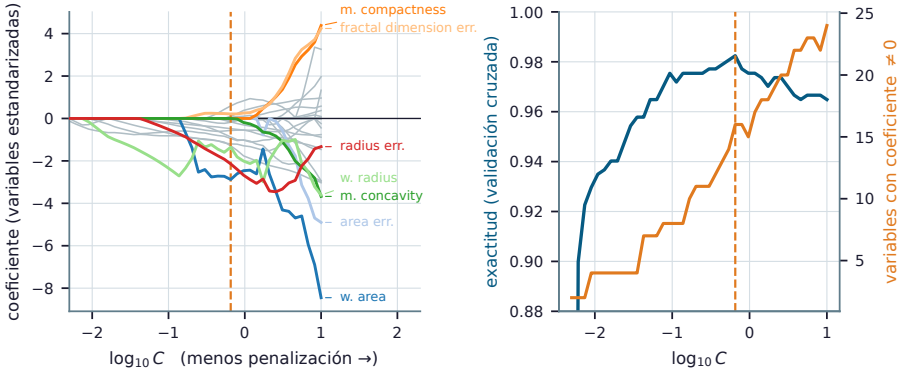


Dos coeficientes β_1, β_2. Las elipses son curvas de igual error (el centro es el mínimo sin penalizar). El área naranja es lo permitido por la penalización.

L1 (rombo): sus *esquinas* están sobre los ejes. La primera elipse en tocarlo suele hacerlo en una esquina: un coeficiente exactamente 0.

L2 (círculo, Ridge): sin esquinas, el contacto ocurre en un punto cualquiera: coeficientes pequeños, pero ninguno exactamente 0.

E3 · Trayectoria de Lasso en el diagnóstico



Regresión logística L1 con las 30 variables estandarizadas. **Izquierda**: cada línea es el coeficiente de una variable al variar C. **Derecha**: exactitud y número de variables con coeficiente distinto de cero. Línea naranja discontinua: $C \approx 0.65$.

Mucha penalización (izquierda, $C \approx 0.005$): solo 2 variables activas y el modelo es muy pobre. Al relajarla, las variables entran *una por una*.

$C \approx 0.65$: 16 variables activas y exactitud 0.982. Con 7 variables ($C \approx 0.05$) ya hay 0.965.

E4 · Lasso en código: SelectFromModel

```python
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

modelo = Pipeline([
  ("sc",  StandardScaler()),   # obligatorio
  ("sel", SelectFromModel(
      LogisticRegression(penalty="l1",
          solver="liblinear", C=0.3))),
  ("lr",  LogisticRegression(max_iter=3000)),
])
modelo.fit(X_ent, y_ent)
modelo[:-1].get_feature_names_out()
```

Cómo funciona

`SelectFromModel` se queda con las variables cuyo coeficiente (en valor absoluto) supera un umbral (por defecto, casi 0 para L1).

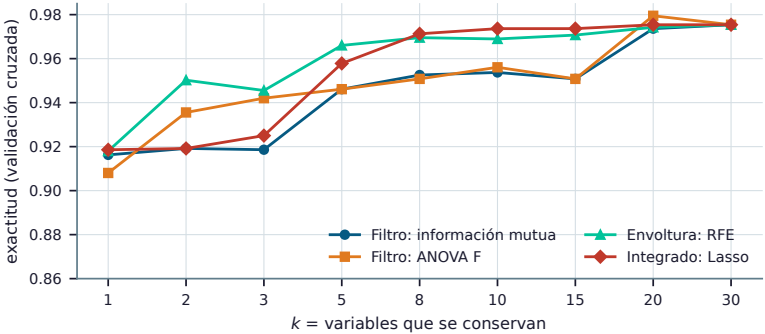
Otros integrados

Árboles y bosques dan `feature_importances_`; sirven con `SelectFromModel`, pero reparten la importancia entre variables correlacionadas.

E5 · Cuidados con Lasso

Cuidado	Qué pasa	Qué hacer
Escala de las variables	la penalización trata igual a todos los coeficientes; una variable en unidades pequeñas necesita un coeficiente grande y se castiga más	estandarizar siempre antes (dentro del `Pipeline`)
Variables correlacionadas	elige una del grupo, casi al azar, y deja las otras en 0; cambia con otra muestra	revisar la estabilidad (sección F); Elastic Net (Unidad 5)
Elegir C o α	cambia cuántas variables sobreviven	validación cruzada (`LogisticRegressionCV`, `LassoCV`)
Coeficientes sesgados	la penalización encoge los coeficientes de las variables que sí se quedan	reajustar el modelo sin penalización con las variables elegidas

F1 · Comparación: las cuatro estrategias con el mismo protocolo



Exactitud de regresión logística según el número k de variables que deja cada método. Toda la selección va **dentro** del pipeline; validación cruzada de 5 particiones, 3 repeticiones.

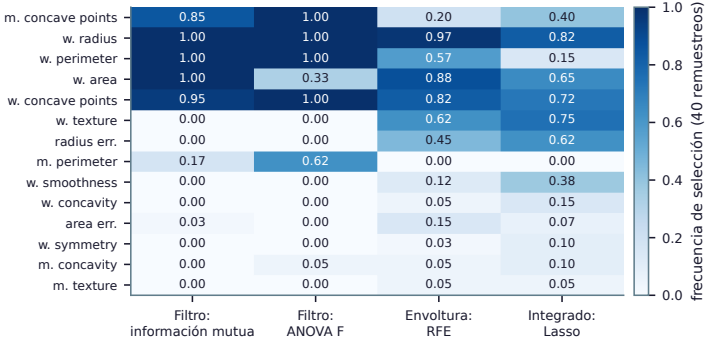
Con pocas variables ($k = 8$ a 10)

RFE y **Lasso** llegan a 0.969–0.974; los filtros, a 0.951–0.956. Eligen variables que se *complementan*.

Con muchas ($k \geq 20$)

Todos convergen a ≈ 0.975: las 30 variables tienen mucha información redundante.

F2 · Estabilidad: ¿elige siempre las mismas variables?



40 repeticiones de la selección de $k = 5$ variables con una muestra bootstrap distinta; cada celda es la frecuencia con que se eligió esa variable.

Filtros

Casi siempre las mismas (frecuencia media de las 5 más elegidas: 0.96 y 0.93; 7 variables distintas en total), pero son redundantes entre sí.

RFE y Lasso

Reparten la elección entre 14 variables distintas (0.78 y 0.72): ante variables correlacionadas, *cualquiera* sirve.

F3 · ¿Qué método usar? Una guía práctica

Situación	Método	Por qué
Miles de variables; primer descarte rápido	Filtro (varianza, MI)	costo muy bajo; sirve para reducir antes de otro método
Variables muy correlacionadas	Filtro por redundancia + Lasso	evita repetir información
Pocos datos, pocas variables, modelo fijo	RFE / RFECV	mira las variables juntas
Modelo lineal con muchas variables	Lasso (con C por validación cruzada)	selecciona y ajusta a la vez
Hay que *explicar* cuáles importan	Lasso o RFE, y revisar estabilidad	la lista debe resistir un remuestreo

Siempre

Dentro del `Pipeline` y de la validación cruzada; comparar contra el modelo con todas las variables; **conocimiento del dominio** antes que automatización: ninguna variable médica se descarta sin que un experto lo valide.

F4 · Comparar selectores con `GridSearchCV`

```
from sklearn.model_selection import GridSearchCV, RepeatedStratifiedKFold
from sklearn.feature_selection import SelectKBest, RFE, SelectFromModel, mutual_info_classif

pipe = Pipeline([("sc", StandardScaler()),
                 ("sel", "passthrough"),      # se cambia en la rejilla
                 ("lr", LogisticRegression(max_iter=3000))])

rejilla = [
  {"sel": [SelectKBest(mutual_info_classif)], "sel__k": [5, 10, 20]},
  {"sel": [RFE(LogisticRegression(max_iter=3000))],
   "sel__n_features_to_select": [5, 10, 20]},
  {"sel": [SelectFromModel(LogisticRegression(penalty="l1", solver="liblinear"))],
   "sel__estimator__C": [0.05, 0.3, 1.0]},
]
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=0)
busq = GridSearchCV(pipe, rejilla, cv=cv, scoring="accuracy").fit(X_ent, y_ent)
print(busq.best_params_, busq.best_score_)
```

El paso `"sel"` puede ser cualquiera de los tres métodos: la rejilla los compara con la misma validación cruzada y la selección se rehace en cada partición.

Receta de ingeniería y selección en siete pasos

1. **Dividir primero** (entrenamiento / prueba) y no tocar la prueba.
2. **Crear** variables con sentido del dominio (razones, log, interacciones); validar cada idea.
3. **Estandarizar** antes de polinomios, RFE y Lasso.
4. **Filtro rápido**: varianza y redundancia ($|r| > 0.9$).
5. **Elegir método**: información mutua / RFE / Lasso, según la guía.
6. **Decidir k o C** con validación cruzada, dentro del `Pipeline`.
7. **Comprobar estabilidad** y tocar la prueba **una vez**.

Las tres ideas de la unidad

1. Crear variables incorpora **conocimiento**; seleccionar quita **ruido y redundancia**.
2. Cada familia de métodos tiene un costo y un tipo de error distinto: no hay un ganador universal.
3. Seleccionar es aprender de los datos: **va dentro de la validación cruzada**.

Actividad: comparar métodos de selección en un conjunto de datos médico

Notebook de la actividad (trabajo individual)

1. Explorar el conjunto de cáncer de mama y fijar una **línea base** con las 30 variables.
2. **Crear** variables del dominio y comprobar si ayudan.
3. Aplicar **filtros**: varianza, correlación con el objetivo, redundancia, información mutua.
4. Aplicar **RFE / RFECV**.
5. Aplicar **Lasso** (`SelectFromModel`).
6. Comparar los cuatro métodos con **validación cruzada** y medir su **estabilidad**.
7. Provocar y corregir una **fuga** al seleccionar variables.

Entregable

Una tabla de comparación:

método | k | **exactitud de validación** | **variables elegidas** | **comentario**

Formato: individual. Al terminar, sube el notebook ejecutado y la tabla.

Tiempo: lo que quede de sesión.



¿Preguntas?

¡Pasemos a la actividad con datos médicos!